# Optional: Sample Project: The Shopping Application

## Estimated Time: 90 minutes

Note: Though this lab is split into multiple parts, it is highly recommended it is all completed in one sitting as the project environment may not be persisted. If for any reason that is not possible, please push your changes to git so that the changes are copied to your remote repository and you can start where you left.

## Objectives:

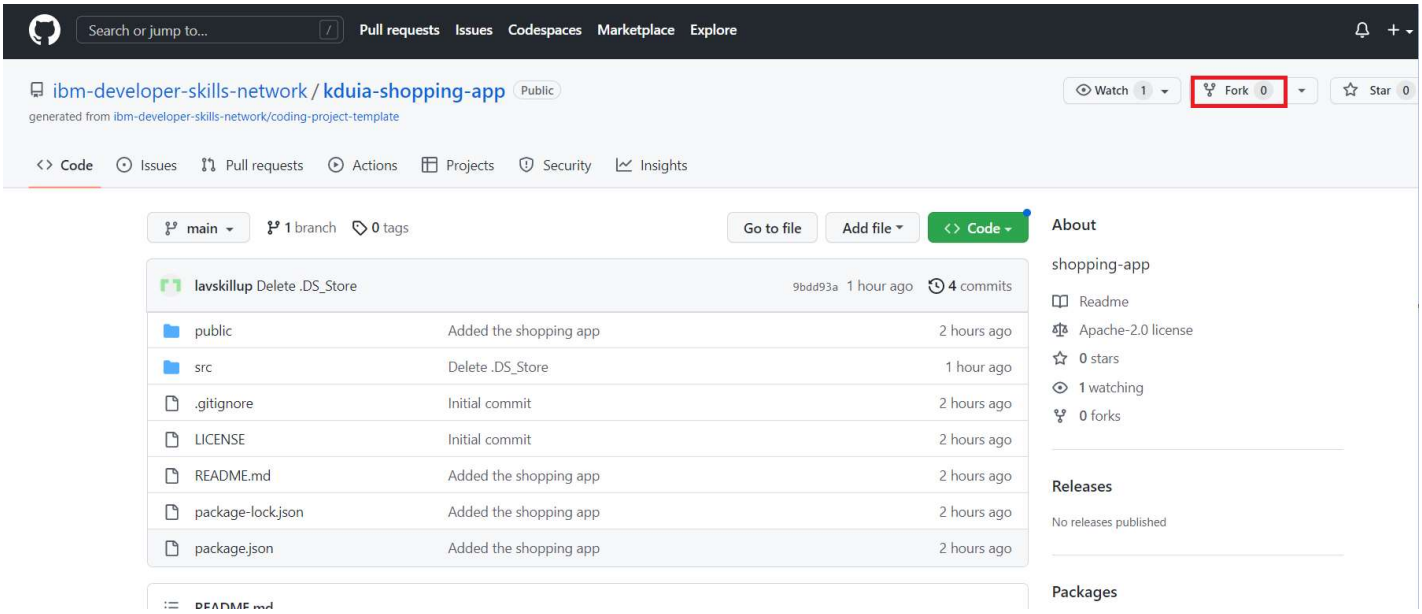
- Setup a React Project.
- Put the UI Components in Place.
- Render the created components and context in App.js.
- View your app on the browser.

## Exercise 1: Setup a React Project

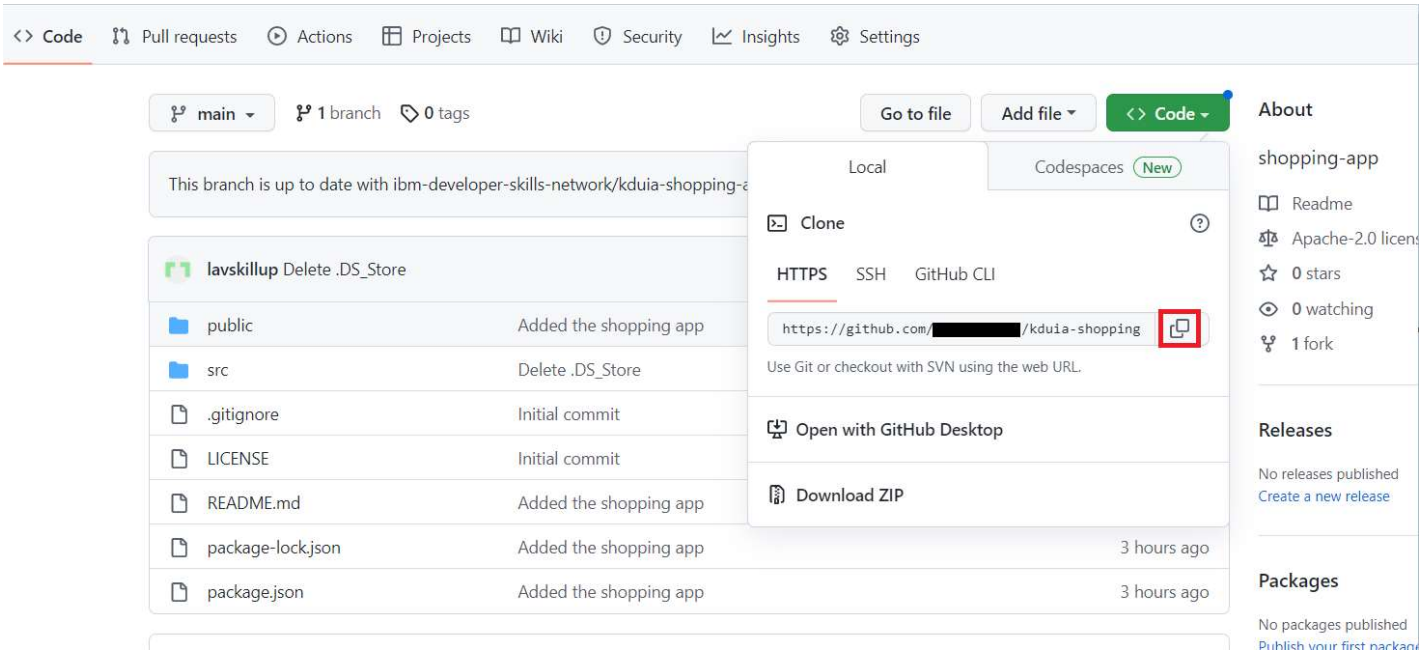
**Fork the Git repository to have the react code you need to start**

**1.** Go to the project repository on this link which has the partially developed code for react code.

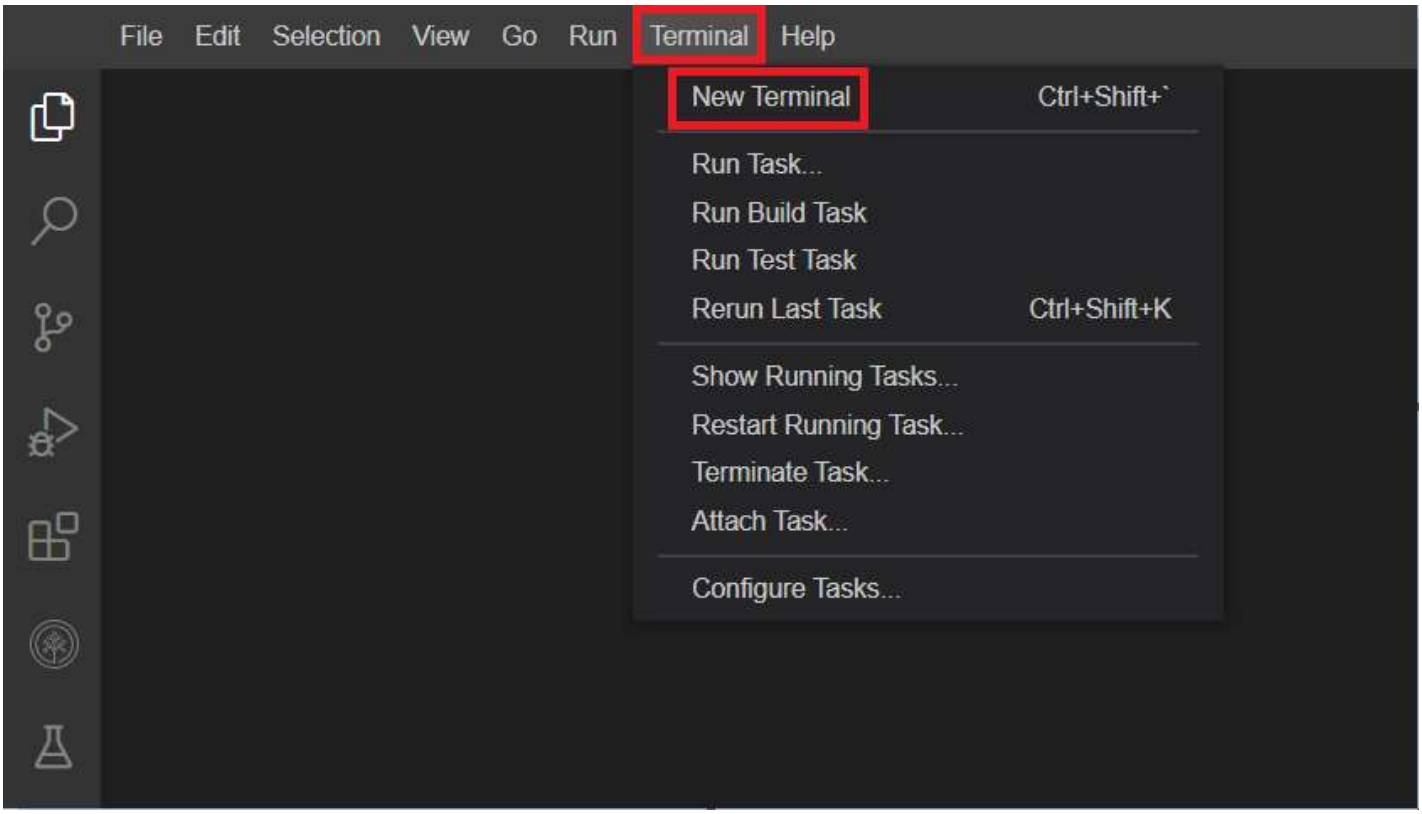
**2.** Create a fork of the repository into your own. *You will need to have a github account of your own to do so.*




**3.** Go to your repository and copy the clone url.



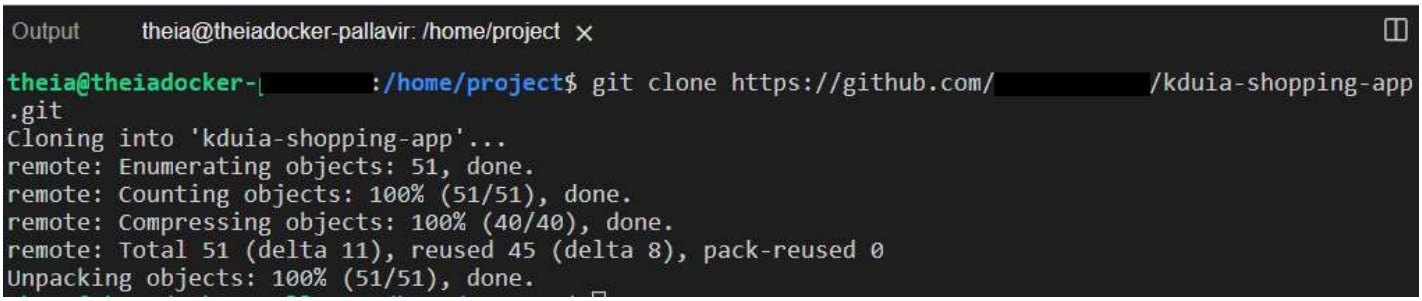

**4.** Open a terminal window by using the menu in the editor: Terminal > New Terminal.




**5.** Clone the repository by running the command given below:

```
1. 1
1. git clone <your_repo_name>
```

Copied!




**6.** This will clone the repository with Shopping application files in your home directory in the lab environment. Check the same with the following command.

```
1
ls
```

Copied! Executed!

**7.** Change to the project directory and check its contents.

```
1
cd kduia-shopping-app && ls
```

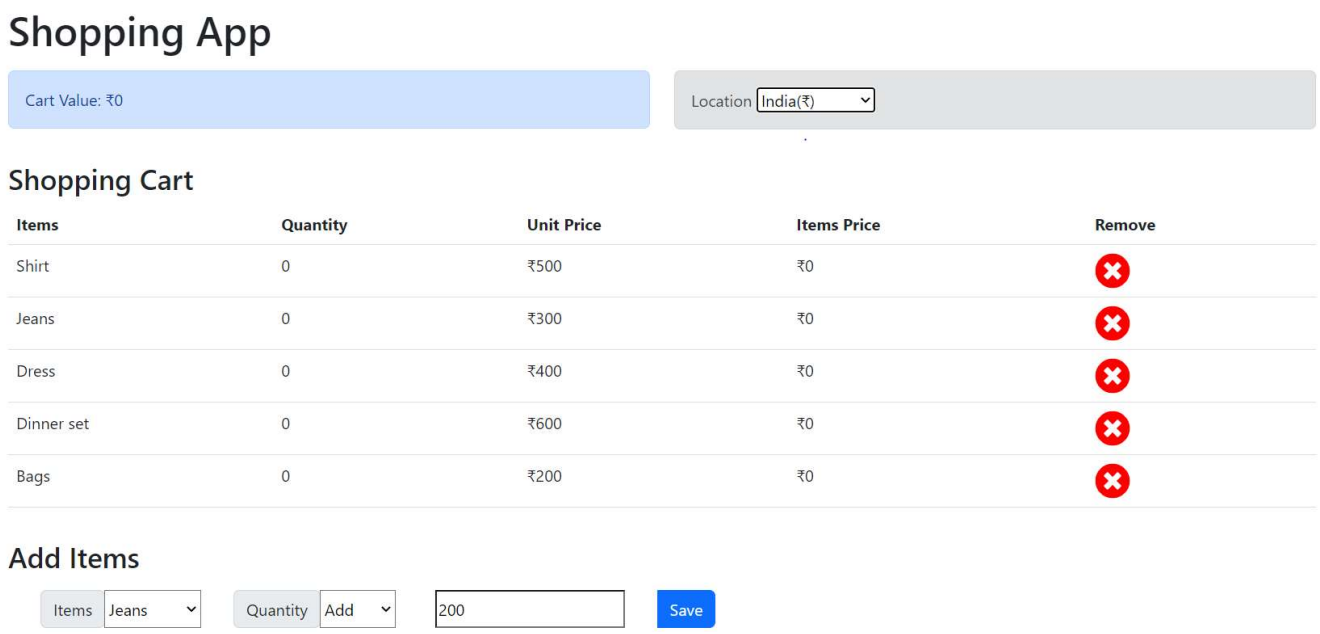
Copied! Executed!

**8.** All packages required to be installed are listed in package.json. Run npm install -s, to install and save those packages.

```
1
npm install  -s
```

Copied! Executed!

## Exercise 2: Put the UI Components in Place.




The image above displays the Shopping application page you will be developing in this sample project. This lab will help you in gaining the knowledge about react to build a shopping app and complete the tasks provided in final project: Budget Allocation application.

It has the following five components:

- CartValue
- ExpenseItem
- ExpenseList
- ItemSelected
- Location

All of these components will be using redux for state management through AppContext.js. You can open the AppContext.js by clicking on the below button.

- In `AppContext.js` you will be creating reducer, which is used to update the state, based on the action. Then you will set the initial state for the Shopping Cart. You will be creating the Provider component which wraps the components you want to give access to the state.

Open **AppContext.js** in IDE

**1.** Make changes to **AppContext.js** component. Open the code, in the `src/context/AppContext.js`. Copy the below code and paste in the AppContext.js

```
1
2
3
4
5
6
7
8
9
10
```

```
11.
12.
13.
14.
15.
16.
17.
18.
19.
20.
21.
22.
23.
24.
25.
26.
27.
28.
29.
30.
31.
32.
33.
34.
35.
36.
37.
38.
39.
40.
41.
42.
43.
44.
45.
46.
47.
48.
49.
50.
51.
52.
53.
54.
55.
56.
57.
58.
59.
60.
61.
62.
63.
64.
65.
66.
67.
68.
69.
70.
71.
72.
73.
74.
75.
76.
77.
78.
79.
80.
81.
82.
83.
84.
85.
86.
87.
88.
89.
90.
91.
92.
93.
94.
95.
96.
97.
98.
99.
100.
```

```
import React, { createContext, useReducer } from 'react';

// 5. The reducer - this is used to update the state, based on the action
export const AppReducer = (state, action) => {
    let new_expenses = [];
    switch (action.type) {
        case 'ADD_QUANTITY':
            let updatedqty = false;
            state.expenses.map((expense)=>{
                if(expense.name === action.payload.name) {
```

```
                    expense.quantity = expense.quantity + action.payload.quantity;
                    updatedqty = true;
                }
                new_expenses.push(expense);
                return true;
            })
            state.expenses = new_expenses;
            action.type = "DONE";
            return {
                ...state,
            };

            case 'RED_QUANTITY':
                state.expenses.map((expense)=>{
                    if(expense.name === action.payload.name) {
                        expense.quantity = expense.quantity - action.payload.quantity;
                    }
                    expense.quantity = expense.quantity < 0 ? 0: expense.quantity;
                    new_expenses.push(expense);
                    return true;
                })
                state.expenses = new_expenses;
                action.type = "DONE";
                return {
                    ...state,
                };
        case 'DELETE_ITEM':
            state.expenses.map((expense)=>{
                if(expense.name === action.payload.name) {
                    expense.quantity = 0;
                }
                new_expenses.push(expense);
                return true;
            })
            state.expenses = new_expenses;
            action.type = "DONE";
            return {
                ...state,
            };
    case 'CHG_LOCATION':
            action.type = "DONE";
            state.Location = action.payload;
            return {
                ...state
            }

        default:
            return state;
    }
};

// 1. Sets the initial state when the app loads
const initialState = {
    expenses: [
        { id: "Shirt", name: 'Shirt', quantity: 0, unitprice: 500 },
        { id: "Jeans", name: 'Jeans', quantity: 0, unitprice: 300 },
        { id: "Dress", name: 'Dress', quantity: 0, unitprice: 400 },
        { id: "Dinner set", name: 'Dinner set', quantity: 0, unitprice: 600 },
        { id: "Bags", name: 'Bags', quantity: 0, unitprice: 200 },
    ],
    Location: '£'
};

// 2. Creates the context this is the thing our components import and use to get the state
export const AppContext = createContext();

// 3. Provider component - wraps the components we want to give access to the state
// Accepts the children, which are the nested(wrapped) components
export const AppProvider = (props) => {
    // 4. Sets up the app state. takes a reducer, and an initial state
    const [state, dispatch] = useReducer(AppReducer, initialState);

    const totalExpenses = state.expenses.reduce((total, item) => {
        return (total = total + (item.unitprice*item.quantity));
    }, 0);
state.CartValue = totalExpenses;

    return (
        <AppContext.Provider
            value={{
                expenses: state.expenses,
                CartValue: state.CartValue,
                dispatch,
                Location: state.Location
            }}
        >
            {props.children}
        </AppContext.Provider>
    );
};
```

Copied!

- In AppContext.js you are setting the initial state of `Expenses` and `Location`. You can see how the items, their respective unit price are all added to Expenses. Here, you are adding an initial expenses, creating a Provider component, setting up the useReducer hook which will hold your state, and allow you to update the state via dispatch.
- Adding a new case to the switch statement called "ADD_QUANTITY", "RED_QUANTITY", "DELETE_ITEM", "CHG_LOCATION".

**2.** Make changes to **CartValue.js** component. You can open the CartValue.js by clicking on the below button.

Open **CartValue.js** in IDE

Copy the below code and paste in the CartValue.js

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
```

```
import React, { useContext } from 'react';
import { AppContext } from '../context/AppContext';

const CartValue = () => {
    const { expenses, Location } = useContext(AppContext);
    const totalExpenses = expenses.reduce((total, item) => {
        return (total += (item.unitprice * item.quantity));
    }, 0);

    return (
        <div className='alert alert-primary'>
            <span>Cart Value: {Location}{totalExpenses}</span>
        </div>
    );
};

export default CartValue;
```

Copied!

- Here, you are importing AppContext from your Context.Import the useContext hook, and pass your AppContext to it - this is how a component connects to the context in order to get values from global state. The Bootstrap Alert classes are used to give a nice gray background by adding some text and hard coding a value.
- Now if you change the budget in AppContext and reload your browser, you will see the budget updates on the UI.

**3.** Make changes to **ExpenseList.js** component. You can open the ExpenseList.js by clicking on the below button.

Open **ExpenseList.js** in IDE

Copy the below code and paste in the ExpenseList.js

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
```

```
import React, { useContext } from 'react';
import ExpenseItem from './ExpenseItem';
import { AppContext } from '../context/AppContext';

const ExpenseList = () => {
    const { expenses } = useContext(AppContext);

    return (
        <table className='table'>
              <thead className="thead-light">
            <tr>
```

```
                <th scope="col">Items</th>
                <th scope="col">Quantity</th>
                <th scope="col">Unit Price</th>
                <th scope="col">Items Price</th>
                <th scope="col">Remove</th>
              </tr>
            </thead>
              <tbody>
              {expenses.map((expense) => (
                  <ExpenseItem id={expense.id} key={expense.id} name={expense.name} quantity={expense.quantity} unitprice={expense.unitprice} />
              ))}
              </tbody>
          </table>
      );
};

export default ExpenseList;
```

Copied!

- In ExpenseList you are importing your AppContext and useContext hook like before. Here, you are creating a list, using the map function to iterate over the expenses, and displaying an ExpenseItem component.

**4.** Make changes to **ExpenseItem.js** component.You can open the ExpenseItem.js by clicking on the below button.

Open **ExpenseItem.js** in IDE

Copy the below code and paste in the ExpenseItem.js

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
```

```
import React, { useContext } from 'react';
import { AppContext } from '../context/AppContext';
import { FaTimesCircle } from 'react-icons/fa';

const ExpenseItem = (props) => {
    const { dispatch, Location} = useContext(AppContext);

    const handleDeleteItem = () => {
        const item = {
            name: props.name,
        };

        dispatch({
            type: 'DELETE_ITEM',
            payload: item,
        });
    };


    return (
        <tr>
        <td>{props.name}</td>
        <td>{props.quantity}</td>
        <td>{Location}{parseInt(props.unitprice)}</td>
        <td>{Location}{parseInt(props.quantity)*parseInt(props.unitprice)}</td>
        <td><FaTimesCircle size='2.2em' color="red" onClick={handleDeleteItem}></FaTimesCircle></td>
        </tr>
    );
};

export default ExpenseItem;
```

Copied!

- In ExpenseItem you are importing dispatch from Context, which allows you to dispatch a delete action. You are creating a function that gets called when the delete icon is clicked.

**5.** Make changes to **ItemSelected.js** component.You can open the ItemSelected.js by clicking on the below button.

Open **ItemSelected.js** in IDE

Copy the below code and paste in the ItemSelected.js

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
```

```
import React, { useContext, useState } from 'react';
import { AppContext } from '../context/AppContext';

const ItemSelected = (props) => {
    const { dispatch} = useContext(AppContext);

    const [name, setName] = useState('');
    const [quantity, setQuantity] = useState('');
    const [action, setAction] = useState('');


    const submitEvent = () => {

        const item = {
            name: name,
            quantity: parseInt(quantity),
```

```
        };

        if(action === "Reduce") {
            dispatch({
                type: 'RED_QUANTITY',
                payload: item,
            });
        } else {
                dispatch({
                    type: 'ADD_QUANTITY',
                    payload: item,
                });
            }
    };

    return (
        <div>
            <div class=Name='row'>

            <div class=Name="input-group mb-3" style={{ marginLeft: '2rem' }}>
                    <div class=Name="input-group-prepend">
                <label class=Name="input-group-text" htmlFor="inputGroupSelect01">Items</label>
                </div>
                  <select class=Name="custom-select" id="inputGroupSelect01" onChange={(event) => setName(event.target.value)}>
                        <option defaultValue>Choose...</option>
                        <option value="Shirt" name="Shirt"> Shirt</option>
                <option value="Dress" name="Dress">Dress</option>
                <option value="Jeans" name="Jeans">Jeans</option>
                <option value="Dinner set" name="Dinner set">Dinner set</option>
                <option value="Bags" name="Bags">Bags</option>
                  </select>

                    <div class=Name="input-group-prepend" style={{ marginLeft: '2rem' }}>
                <label class=Name="input-group-text" htmlFor="inputGroupSelect02">Quantity</label>
                </div>
                  <select class=Name="custom-select" id="inputGroupSelect02" onChange={(event) => setAction(event.target.value)}>
                  <option defaultValue value="Add" name="Add">Add</option>
                <option value="Reduce" name="Reduce">Reduce</option>
                  </select>
                  <span class=Name="eco" style={{ marginLeft: '2rem', marginRight: '8px'}}></span>

                    <input
                        required='required'
                        type='number'
                        id='cost'
                        value={quantity}
                        style={{size: 10}}
                        onChange={(event) => setQuantity(event.target.value)}>
                        </input>

                    <button class=Name="btn btn-primary" onClick={submitEvent} style={{ marginLeft: '2rem' }}>
                        Save
                    </button>
                </div>
                </div>

        </div>
    );
};

export default ItemSelected;
```

Copied!

- In ItemSelected, you are importing AppContext and useContext as usual and getting dispatch from your global state. You are creating an event to reduce or add quantity.You are dispatching an action, with a type and your payload. The type tells the reducer how to update the state.

**6.** Make changes to **Location.js** component. You can open the Location.js by clicking on the below button.

Open **Location.js** in IDE

Copy the below code and paste in the Location.js

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
```

```
25
26
27
28
```

```
import React, { useContext } from 'react';
import { AppContext } from '../context/AppContext';

const Location = () => {
  const {dispatch } = useContext(AppContext);

    const changeLocation = (val)=>{
            dispatch({
                type: 'CHG_LOCATION',
                payload: val,
            })
    }


  return (
        <div className='alert alert-secondary'> Location {
      <select name="Location" id="Location" onChange={event=>changeLocation(event.target.value)}>
        <option value="£">Uk(£)</option>
        <option value="₹">India(₹)</option>
        <option value="€">Europe(€)</option>
        <option value="CAD">Canada(CAD)</option>
      </select>
      }
    </div>
    );
};

export default Location;
```

Copied!

- In Location.js, you are importing AppContext and adding changeLication class to change the location along with its currencies. When you change the Location, currencies will be updated at all the required places.

## Excercise 3: Render the created components and context in App.js

**1.** Make changes to **App.js**. Open the code, in the `src/App.js`. You need to add the code in the container div to import and add the components created above.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
```

```
import React from 'react';
import 'bootstrap/dist/css/bootstrap.min.css';
import { AppProvider } from './context/AppContext';
import CartValue from './components/CartValue';
import ExpenseList from './components/ExpenseList';
import ItemSelected from './components/ItemSelected';
import Location from './components/Location';

function App() {
  return (
    <div className="App">
    </div>
  );
}

export default App;
```

Copied!

▶ Click here to view the hint
▶ Click here to view the solution

- Here, you are importing different components,adding a bootstrap container that helps you center your App on the page. Importing your AppProvider and nested your components in the AppProvider element.

## Exercise 4: Launch and view your react app on the browser

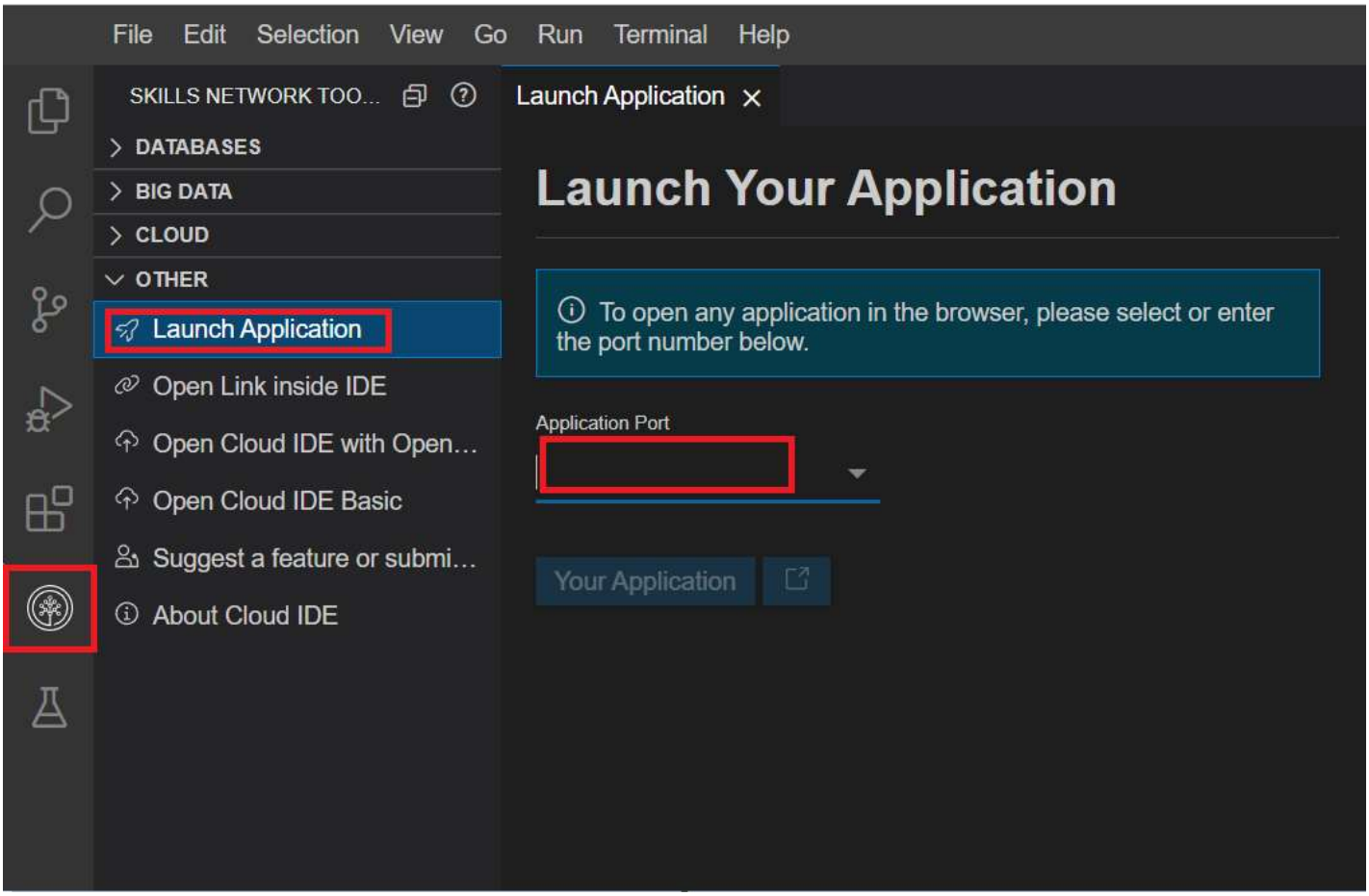
**1.** Make sure you are in the `kduia-shopping-app` directory and run the server using the following command.

```
1
```

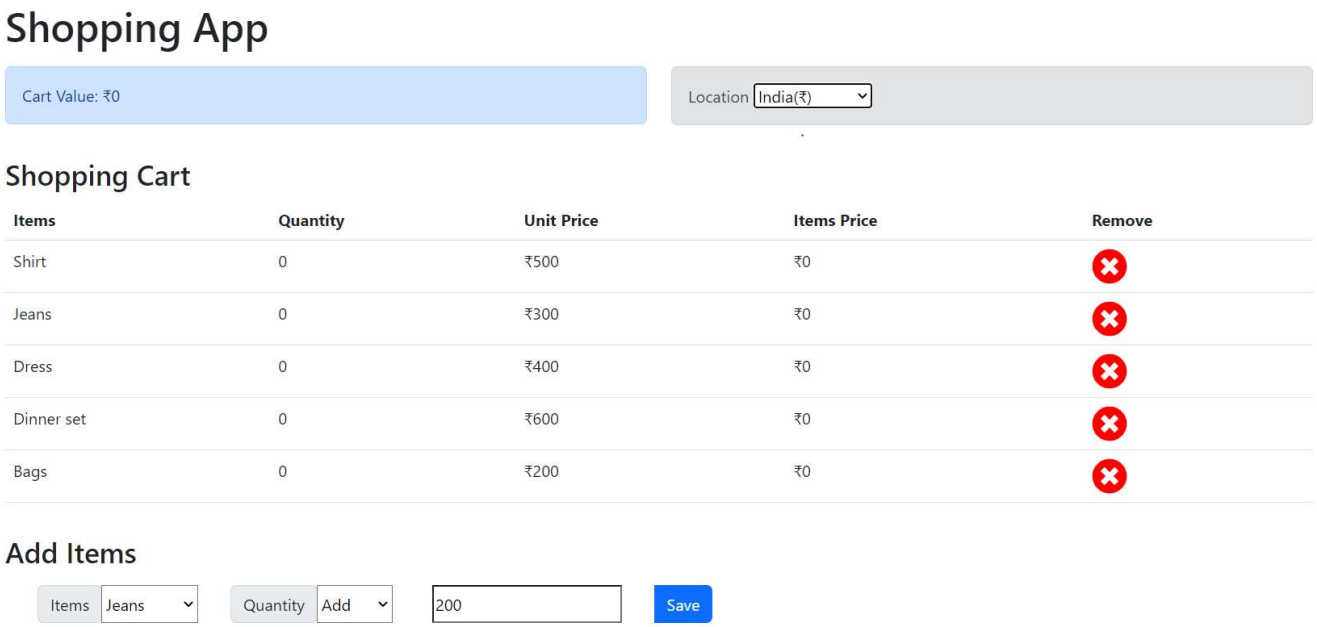
```
npm start
```

Copied!

**2.** Click on the Skills Network button on the left, it will open the **"Skills Network Toolbox"** . Then click the **Other** then **Launch Application**. From there you should be able to enter the port 3000.




Verify your output on the browser.




Congratulations! You have completed this sample shopping application lab and know how to create components of a Shopping application.

## Author(s)

Pallavi Rai

**Change Log**

| Date | Version | Changed by | Change Description |
|---|---|---|---|

| Date | Version | Changed by | Change Description |
|---|---|---|---|
| 2022-08-31 | 1.0 | Pallavi Rai | Initial version created |

**© IBM Corporation 2022. All rights reserved.**